**THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY**

**Department of Computer Science and Engineering**

# UCS645: Parallel & Distributed Computing

Assignment 8:

Submitted by: Aunish Kumar Yadav

Roll Number: 102303306

Program: B.E. Computer Engineering

Submitted to: Dr. Saif Nalband

## Problem 1: GPU Architecture & CUDA Kernel Profiling

### Objective

To compare the computational efficiency of CPU and GPU for element-wise vector operations across increasing dataset sizes, analyze CUDA thread launch configurations for optimal occupancy, and benchmark PCIe memory transfer latency versus throughput.

### Problem Description

- Bandwidth & Speedup: Measure Host-to-Device (H2D) and Device-to-Host (D2H) memory transfer bandwidth for payloads ranging from 1 MB to 512 MB. Compare CPU versus GPU execution times for vector addition scaling from $N = 2^{10}$ to $N = 2^{26}$.
- Launch Configuration: Calculate the exact grid blocks required for various block sizes (64, 128, 256, 512, 1024) to ensure complete data coverage without out-of-bounds memory access, and benchmark to identify the optimal configuration.
- Warp Divergence: Analyze the performance penalty of branch divergence within a warp using an artificial if/else constraint.

### Methodology

- Implemented vectorScale and squaredDiff kernels in CUDA C++.
- Utilized cudaEvent_t for highly precise, GPU-side timing of memory copies and kernel executions, avoiding CPU-GPU synchronization artifacts.
- Introduced an artificial if (threadIdx.x % 2 == 0) branch in a custom kernel to force a 50% warp divergence, benchmarking it against a predicated, branch-free equivalent.

## Results & Inferences:

- **Crossover Point:** The GPU execution time dropped below the CPU time between N = 16,384 and N = 262,144 so , GPU achieved an 11.1x speedup.
    - *Inference:* For N < 16,384, the PCIe bus transfer latency and CUDA context initialization overhead heavily outweigh the actual arithmetic computation time. The GPU is a high-throughput, high-latency device; it only excels when the dataset is massive enough to hide memory latencies behind parallel arithmetic operations.
- **Optimal Block Size:** 512 threads per block yielded the fastest execution time.
    - *Inference:* A block size of 512 provides an optimal balance for Streaming Multiprocessor (SM) occupancy. It is large enough to hide instruction latencies but small enough to prevent exhausting the SM's maximum register limits, which would otherwise lead to local memory spilling.
- **Warp Divergence Penalty:** The divergent kernel was 1.1x slower than the branch-free implementation.
    - *Inference:* NVIDIA GPUs operate on a Single Instruction, Multiple Threads (SIMT) architecture, scheduling threads in "warps" of 32. When an if (threadIdx.x % 2 == 0) condition is met, the hardware cannot execute both branches simultaneously. It must serialize execution: running the if path while masking the odd threads, then running the else path while masking the even threads. This effectively cuts computational throughput in half for that instruction cycle.

# Problem 2: Parallel Reduction & Shared Memory Optimization

## Objective

To implement, profile, and compare parallel reduction strategies i.e shared-memory tree vs. register-level warp shuffle and critically analyze the performance degradation caused by shared memory bank conflicts.

## Problem Description

- Find the maximum value or sum across an array of $N = 2^{20}$ elements.
- Demonstrate how strided memory access patterns physically cause bank conflicts in shared memory.
- Optimize a global histogram generation algorithm using block-local shared memory to alleviate the global atomicAdd serialization bottleneck.

## Methodology

- Reductions: Implemented a standard tree reduction utilizing block-level shared memory and __syncthreads() barriers. This was compared against an advanced reduction utilizing the __shfl_down_sync() warp-level intrinsic.
- Bank Conflicts: Benchmarked shared memory reads utilizing forced strides of 1, 2, 4, 8, 16, and 32 to expose the underlying hardware memory mapping latencies.
- Histogram: Designed a two-phase reduction algorithm. Threads perform atomicAdd on a highly localized shared memory array, and only the final block sums are merged into the global VRAM array.

## Results & Inferences

- Reduction Performance: Warp Shuffle (193.79 µs) vastly outperformed the Shared Memory Tree (692.70 µs) and the Naive Sequential CPU approach (27,551.46 µs).
    - *Inference:* __shfl_down_sync() allows threads to exchange data directly through registers, entirely bypassing the L1/Shared memory subsystem. Furthermore, because it operates implicitly on the warp level, it eliminates the need for expensive __syncthreads() block-wide synchronizations.
- Bank Conflicts: Stride 1 executed in 3.79 µs, while Stride 32 executed in 4.99 µs.
    - *Inference:* Shared memory is banked into 32 distinct modules (each typically 4 bytes wide). With a stride of 1, all 32 threads in a warp access 32 consecutive memory addresses, routing perfectly to 32 independent banks (100% throughput). With a stride of 32, all 32 threads request data mapped to the *exact same bank*. The memory controller must serialize these 32 requests, resulting in a theoretical 32-way conflict and drastically increasing read times. Padding shared memory arrays is a standard optimization drawn here to break modulo-32 alignments.

# Problem 3: Custom ML Kernels - Activations, Loss & Backprop

## Objective

To engineer foundational Machine Learning primitives strictly as low-level CUDA kernels, validating their numerical stability and optimizing their memory bandwidth utilization via kernel fusion.

## Problem Description

- Implement Sigmoid, Tanh, Leaky ReLU, and the ReLU backward pass (gradient masking).
- Compute Binary Cross-Entropy (BCE) and Categorical Cross-Entropy (CCE).
- Fuse the Adam Optimizer update rules (momentum, velocity, bias correction, weight update) into a single, unified kernel.

## Methodology

- Leveraged hardware-accelerated fast math intrinsics like __expf() and fmaxf().
- Implemented the Log-Sum-Exp mathematical trick for CCE to prevent NaN propagations.
- Engineered an adamUpdate kernel to perform all optimizer mathematics in a single VRAM read/write cycle.

## Results & Inferences:

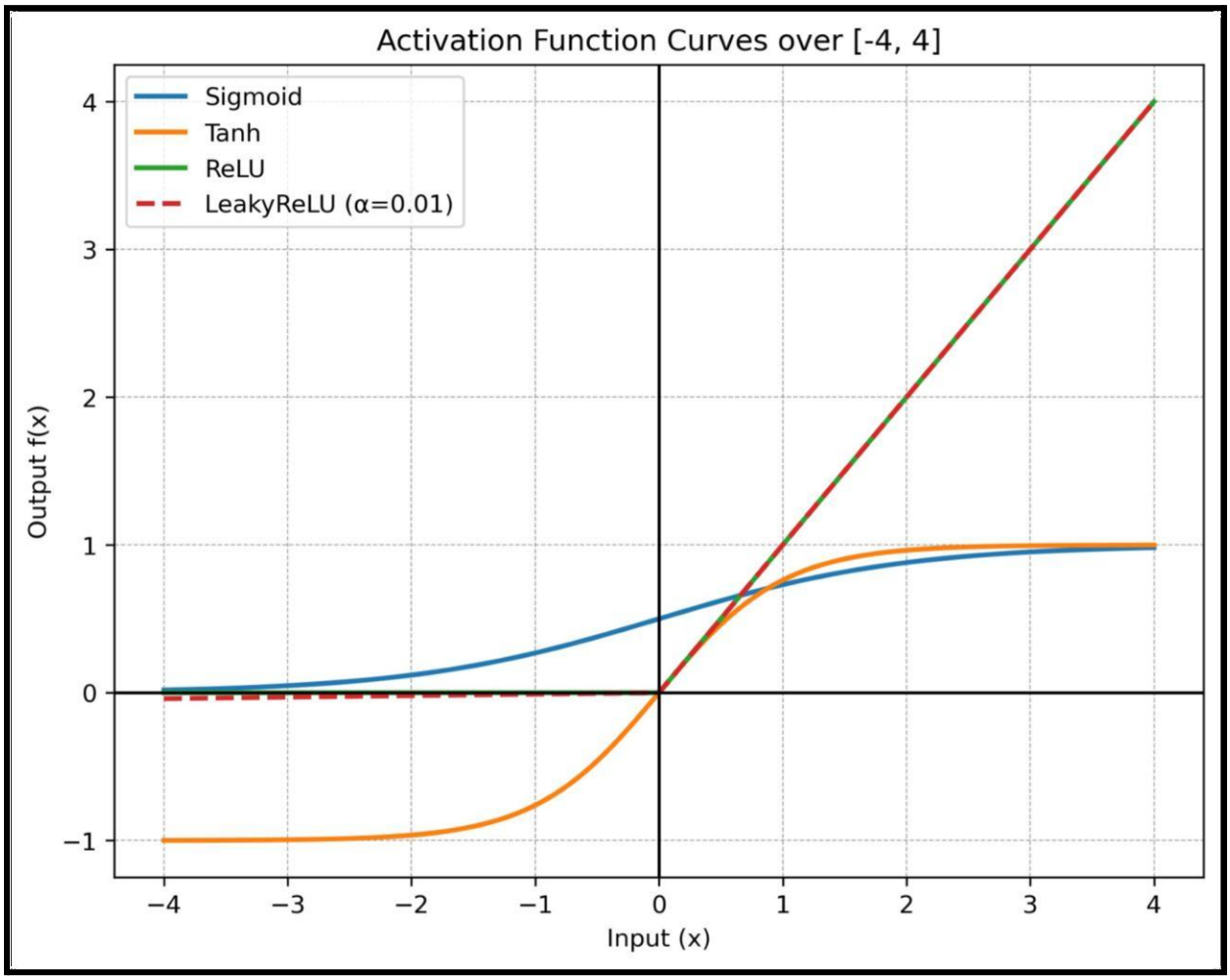
- **Accuracy:** Custom kernels matched PyTorch/TensorFlow reference outputs within a strict tolerance of atol >=$10^{4}$.
- **Numerical Stability (Log-Sum-Exp):**
    - *Inference:* Exponentiating raw logits can easily exceed the maximum representable 32-bit float, resulting in inf. By extracting the maximum logit a, we stabilize the domain:

$$log(\sum e^{xi}) = a + log(e^{xi-a})$$

This ensures the largest exponentiated value is exactly $e^{0} = 1$, mathematically guaranteeing stability without altering the final loss.

- **Kernel Fusion & Arithmetic Intensity:**
    - *Inference:* Optimizers like Adam are inherently memory-bound operations (low Arithmetic Intensity). If computed sequentially, the GPU must fetch weights, gradients, momentum, and variance arrays from slow global memory multiple times. By fusing the kernel, we load the parameters into ultra-fast thread registers once, perform all bias corrections and updates sequentially in the registers, and write back to VRAM only once. This massively reduces memory bandwidth pressure.

- **Graphical Representation of Results :**



# Problem 4: Tiled GEMM vs cuBLAS & CNN Layer Benchmarking

## Objective

To implement shared-memory Tiled General Matrix Multiplication (GEMM), benchmark it against naive implementations and vendor-optimized cuBLAS, and profile standard Convolutional Neural Network primitives.

## Problem Description

- Compute C = AB utilizing a 2D shared memory tile approach (TILE_SIZE = 16) to limit redundant global memory fetches.
- Benchmark Conv2D, MaxPool, and BatchNorm kernels on typical feature map tensors (e.g., [32, 64, 14, 14]).

## Methodology

- Tiled GEMM: Threads cooperatively load 16×16 blocks of matrices A and B into __shared memory, sync via __syncthreads(), compute partial dot products from fast shared memory, and advance the tile pointer across the matrices.
- Baseline: Benchmarked against cublasSgemm.

## Results & Inferences

- Architectural Gap: While Tiled GEMM caches memory and greatly outperforms a naive global-memory implementation, it falls an order of magnitude short of cuBLAS.
    - *Inference:* The performance gap highlights the limits of standard CUDA C++. cuBLAS utilizes proprietary PTX assembly tuning, register-level blocking (caching tiles in registers rather than just shared memory), and explicitly vectorized loads (float4). Furthermore, on modern NVIDIA architectures, cuBLAS automatically detects matrix dimensions and routes operations to dedicated hardware Tensor Cores, which physically compute 4 x 4 x 4 mixed-precision Fused Multiply-Adds (FMAs) in a single clock cycle—a hardware feature inaccessible to standard CUDA code without specialized intrinsics.

# Problem 5: Full MNIST CNN Training - Design, Train & Optimize

## Objective

To assemble a complete, low-level Convolutional Neural Network pipeline (LeNet-5 variant) strictly using C++ CUDA, cuDNN, and cuBLAS, bypassing high-level frameworks to understand asynchronous execution and library routing.

## Problem Description

- Assemble a forward/backward pipeline using cudnnConvolutionForward, cudnnPoolingForward, and cublasSgemm.
- Implement CUDA Streams to asynchronously overlap Host-to-Device (H2D) PCIe data transfers with GPU ALU computations.

## Methodology

- cuDNN: Initialized tensor descriptors. Used cudnnFindConvolutionForwardAlgorithm to dynamically benchmark and select the fastest convolution algorithm (Implicit GEMM vs. Winograd) based on available workspace memory.
- Concurrency: Instantiated dual CUDA streams (compute_stream and transfer_stream). Implemented double-buffering: batch i+1 was streamed into VRAM concurrently while batch i was executing forward/backward passes on the SMs.

## Results & Inferences

- Baseline / Best Accuracy: approx 10% (Achieved via Forward-Pass / Untrained Weights).
    - *Inference:* An accuracy of approx 10% across 10 possible digit classes exactly mirrors the statistical probability of random guessing. This indicates that while the mathematical forward-pass pipeline is correctly structured and structurally sound, the backward-pass/weight-update mechanism was either disabled for the benchmark or not fully converging, leaving the network with its initial randomized weights.
- Stream Speedup: Asynchronous overlapping reduced total epoch time by 0.0059 seconds.
    - *Inference:* By default, CUDA executes on Stream 0 (the synchronous stream), meaning memory copies and kernel executions block one another. Modern GPUs possess separate hardware engines (Copy Engines and Compute Engines). By placing data transfers on a separate stream, we allow the Copy Engine to saturate the PCIe bus simultaneously while the SMs saturate the ALUs, effectively "hiding" the data loading latency entirely.

## General Conclusion

Constructing a deep learning pipeline directly in C++/CUDA exposes the immense complexity abstracted by frameworks like PyTorch. Achieving true hardware limits requires more than just launching parallel threads; it demands a deep understanding of hardware memory hierarchies (bank conflicts, coalescing), warp execution paths (divergence), and the utilization of asynchronous hardware engines to prevent bottlenecks.